

P0009r3 : Polymorphic Multidimensional Array Reference

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0009r3
Date: 2016-10-14
Reply-to: hcedwar@sandia.gov, balelbach@lbl.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Bryce Lelbach
Contact: balelbach@lbl.gov
Author: Christian Trott
Contact: crtrott@sandia.gov
Author: Mauro Bianco
Contact: mbianco@cscs.ch
Author: Robin Maffeo
Contact: Robin.Maffeo@amd.com
Author: Ben Sander
Contact: ben.sander@amd.com
Audience: Library Evolution Working Group (LEWG)
URL: https://github.com/kokkos/array_ref/blob/master/proposals/P0009.rst

1 Description

The proposed polymorphic multidimensional array reference (`array_ref`) defines types and functions for mapping indices from a multidimensional index space (the domain) to members of a contiguous span of objects (the codomain). This **layout mapping** is one *property* of the `array_ref` that may be specified through a template parameter. The intent is that *properties** are an extensible set of options for multi-index mapping and member access. For example, bounds checking the input multi-index versus the multidimensional extents or accessing members through an atomic interface. The recent Accessors paper (P0367) introduces a rich set of potential access properties.

1.1 Multidimensional array, not array-of-array-of-array-of-array...

The multidimensional array abstraction has been fundamental to numerical computations for over five decades. However, the C/C++ language provides only a one dimensional array abstraction which can be composed into array-of-array-of-array... types. While such types have some similarity to multidimensional arrays they do not provide adequate multidimensional array functionality of this proposal. Two critical functionality differences are (1) multiple dynamic extents and (2) polymorphic mapping of multi-indices to member objects.

1.2 Relationship to P0454 : Wording for a Minimal `mdspan`

P0454, Wording for a Minimal `mdspan`, includes a minimal version of the polymorphic multidimensional array reference capability that aligns with `span` as proposed in P0122.

Assuming acceptance of P0454 our intent is for a subsequent version of this paper to propose extensions to `mdspan` for layout extensibility and other essential properties.

Note that the `array_ref` name in this paper is not entirely satisfactory and the following alternatives were brainstormed during prior LEWG reviews.

LEWG 2015-10-22	LEWG 2016-06-25
view , span , array_ref , slice , array_view , ref , array_span , basic_span , object_span , field	sci_span , numeric_span , multidimensional_span , multidim_span , md_span , vla_span , multispan , multi_span

When P0454 is accepted the name will become `mdspan`.

1.3 Alternative mechanisms for specifying the domain multi-index space

Two potential mechanisms for specifying the domain multi-index space are possible.

(A) The preferred mechanism is concise and aligns with language syntax for specifying multidimensional array extents. However, the preferred mechanism requires a trivial language change to relax the specification for *incomplete* array types to support multiple dynamic extents. This trivial language change is proposed in P0332.

```
using A_type = std::array_ref< int[][8][3] > ;

// standard array traits extracted from data type

static_assert( std::rank< int[][8][3] >::value == 3 );
static_assert( std::extent< int[][8][3] , 0 >::value == 0 );
static_assert( std::extent< int[][8][3] , 1 >::value == 8 );
static_assert( std::extent< int[][8][3] , 2 >::value == 3 );
```

(B) The undesirable mechanism requires an additional extents and dyn property symbols. This mechanism is more verbose and requires the introduction of a *magic* integral value to denote a dynamic extent.

```
// array traits given through new array traits types

using A_type std::array_ref< int , std::extents<std::dyn,8,3> > ;
```

Common Result

The array_ref type resulting from either specification mechanism has the same interface.

```
int * buffer = /* buffer to span of integers */ ;

A_type A( buffer , N );

// static rank and extents

static_assert( A_type::rank() == 3 );
static_assert( A_type::static_extent(0) = 0 );
static_assert( A_type::static_extent(1) = 8 );
static_assert( A_type::static_extent(2) = 3 );

// runtime extents

assert( A.extent(0) = N );
assert( A.extent(1) = 8 );
assert( A.extent(2) = 3 );

// member access through operator()

assert( & A(0,0,0) == buffer );
```

2 Multidimensional Array and Subarray

2.1 Header <array_ref> synopsis: array_ref and subarray

```
namespace std {
namespace experimental {

    template< typename DataType , typename ... Properties >
    class array_ref ;

    // return type of subarray free function is an array_ref
    template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
    // for exposition only:
    detail::subarray_deduction_t< array_ref<DataType,Properties...,SliceSpecifiers...>
    subarray( array_ref< DataType, Properties ... > const & , SliceSpecifiers ... ) noexcept;

    // tag supporting subarray
    struct all_type {};
    inline constexpr all_type all = all_type{};
}}

```

The array_ref class maps a multi-index within a multi-index space (the domain) to a reference to an object within a span of objects (the codomain).

The subarray free function generates an array_ref with a domain contained within the input array_ref domain and codomain contained within the input array_ref codomain.

The alias detail::subarray_deduction_t class is not proposed and only appears for exposition. An implementation metafunction of this form is necessary to deduce the specific array_ref return type of the subarray function.

2.2 template class array_ref

```
namespace std {
namespace experimental {

template <typename DataType, typename... Properties>
class array_ref {
public:
    // domain and codomain types

    using value_type = typename remove_all_extents<DataType>::type ;
    using pointer     = value_type * ;
    using reference   = /* implementation deduces from value_type and Properties... */ ;
    using size_type   = /* implementation deduces from Properties... */ ;
    using layout      = /* implementation deduces from Properties... */ ;

    // Standard constructors, assignments, and destructor

    ~array_ref() noexcept ;

    constexpr array_ref() noexcept;
    constexpr array_ref(array_ref&&) noexcept ;
    constexpr array_ref(array_ref const&) noexcept ;
    array_ref& operator=(array_ref&&) noexcept ;
    array_ref& operator=(array_ref const&) noexcept ;

    // Constructor and assignment for assignables

    template <typename UType, typename ... UProp>
    constexpr array_ref( array_ref<UType, UProp...> const& ) noexcept;

    template <typename UType, typename ... UProp>
    array_ref& operator=( array_ref<UType, UProp...> const& ) noexcept;

    // Wrapping constructors

    template <typename... IntegralExtent>
    explicit constexpr array_ref(pointer, IntegralExtent... ) noexcept;

    explicit constexpr array_ref(pointer, layout const&) noexcept;

    // mapping domain multi-index to access codomain member

    template <typename... IntegralIndex>
    reference operator()(IntegralIndex...) const noexcept;

    template <typename IntegralIndex>
    reference operator[]( IntegralIndex ) const noexcept; // requires rank() == 1

    // observers of domain: [0..extent(0))X[0..extent(1))X...X[0..extent(rank()-1))

    static constexpr int rank() noexcept;
    static constexpr int rank_dynamic() noexcept;

    static constexpr size_type static_extent(int) noexcept;

    constexpr size_type extent(int) const noexcept;

    constexpr size_type size() const noexcept;

    // observers of the codomain: [data()..data()+span())

    constexpr pointer data() const noexcept;

    constexpr size_type span() const noexcept;

    template <typename... IntegralExtent>
    static constexpr size_type required_span( IntegralExtent ... ) noexcept;

    static constexpr size_type required_span( layout const & ) noexcept;

    // observers of the mapping : domain -> codomain

    static constexpr bool is_always_unique      = /* deduced */ ;
```

```
static constexpr bool is_always_contiguous = /* deduced */ ;
static constexpr bool is_always_strided    = /* deduced */ ;

constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() noexcept;

constexpr size_type stride(int) const noexcept;
};

}}
```

2.2.1 Template arguments

```
template <typename DataType, typename... Properties> class array_ref
```

(A) Preferred Extent Specification Mechanism

DataType

Requires: Is a complete or incomplete array type (8.3.4.p3). Each omitted static extent in the incomplete array type, [], denotes a *dynamic* extent.

Effects: The value type, domain index space rank, static extents, and identification of dynamic extents is determined from the possibly incomplete array type DataType .

```
value_type is std::remove_all_extents<DataType>::type ;
```

```
rank() is std::rank<DataType>::value
```

```
static_extent(i) is std::extent<DataType,i>::value
```

A dynamic extent is indicated when `std::extent<DataType,i>::value == 0`

Properties...

Requires: `is_array_property_v< Properties >` for each member of the pack.

Effects: The domain to codomain reference mapping is determined by the content of the property pack.

(B) Undesirable Extent Specification Mechanism

DataType

Requires: Is a non-array type denoting the value type of the array.

Properties...

Requires: `is_array_property_v< Properties >` for each member of the pack.

Effects: The domain index space rank, static extents, and identification of dynamic extents is determined from the `extents` member of the property pack. The domain to codomain mapping is determined by the remaining members of the property pack

2.2.2 Fundamental Types

```
using size_type = /* implementation defined */ ;
```

Return type for extents and storage type for dynamic extents.

```
using value_type = typename remove_all_extents<DataType>::type ;
```

```
using pointer = value_type * ;
```

Type of the codomain member objects referenced by the array.

```
using reference = /* deduced from DataType and Properties... */ ;
```

Reference type for member access. Unless modified to support special access properties this is `value_type &`. Special access properties may cause `reference` to be a proxy type.

2.2.3 Mapping domain multi-index to access members in the codomain

```
template< typename ... IntegralIndex >
```

```
reference operator()( IntegralIndex ... indices ) const noexcept
```

Requires: `indices` is a multi-index in the domain:

- `conjunction<is_integral<IntegralIndex>::value...>::value`
- `rank() <= sizeof...(IntegralIndex)`
- The `ith` coordinate of `indices...`, denoted as `indices[ith]`, is in the domain: `0 <= indices[ith] < extent(ith)`.
- [Note: Because `extent(ith) == 1` for `rank() <= ith` then extra zero-value indices are valid. --end note]

Returns: A reference to the member object mapped to by `indices...`

Remark: Optimization of the mapping operator is a critical feature of the multidimensional array implementation. Recommended optimizations include:

- Rank-specific overloads to better enable optimization of the member access operator.
- Inlining of a `constexpr` multi-index mapping expression that is **not** included in an optimizer's inlining budget.
- Compile-time evaluation statically determined portions of multi-index mapping expression.
- Deferring promotion of an `IntegralIndex` until evaluation of the multi-index mapping expression.

```
template< typename IntegralIndex >
reference operator[]( IntegralIndex index ) const noexcept
```

Requires: `is_integral<IntegralIndex>::value.rank() == 1`. `0 <= i < extent(0)`.

Returns: A reference to the member object referenced by `index`.

Requires: `0 <= index < extent(0)`

2.2.4 Domain

```
static constexpr int rank() noexcept ;
constexpr size_type extent(int r) const noexcept ;
```

Requires: `0 <= r`

Returns: Rank and extents of the domain where the domain is the Cartesian product of the extents: `[0..extent(0)) X [0..extent(1)) X ... X [0..extent(rank()-1))`. If `rank() <= r` then `extent(r) == 1`.

```
static constexpr size_type static_extent(int r) const noexcept ;
```

Requires: `0 <= r`

Returns: If `0 <= r < rank()` the statically declared extent. A statically declared extent of 0 denotes that the extent is dynamic. If `rank() <= r` then `static_extent(r) == 1`.

```
constexpr size_type size() const noexcept ;
```

Returns: product of extents.

```
static constexpr int rank_dynamic() noexcept ;
```

Returns: number of extents that are dynamic.

2.2.5 Codomain

Not all members of the codomain may be accessible through the layout mapping; i.e., the range of the mapping is contained within the codomain but may not be equal to the codomain.

```
constexpr pointer data() const noexcept ;
constexpr size_type span() const noexcept ;
```

Returns: The codomain is `[ data() .. data() + span() )`

```
template< typename ... IntegralExtent >
static constexpr size_type required_span( IntegralExtent ... dynamic_extent ) noexcept
static constexpr size_type required_span( layout const & ) noexcept
```

Requires: `conjunction<is_integral<IntegralExtent>::value...>::value`. Each `dynamic_extent` is non-negative.

Returns: Required length of contiguous span of objects input the *wrapping constructor* with the corresponding extent argument.

2.2.6 Mapping : domain -> codomain

```
using layout = /* deduced from Properties... */ ;
```

Identification of the layout mapping. If `Properties...` does not include a layout property then layout is `layout_right` denoting the traditional C/C++ mapping.

```
static constexpr bool is_always_unique =
constexpr bool is_unique() const noexcept ;
```

A layout mapping is *unique* if each multi-index in the domain is mapped to a unique member in the codomain.

```
static constexpr bool is_always_contiguous =
constexpr bool is_contiguous() const noexcept ;
```

A layout mapping is *contiguous* if the layout mapping can access every member of the codomain.

A layout mapping that is both unique and contiguous is *bijective* and has `size() == span()`.

```
static constexpr bool is_always_strided =
constexpr bool is_strided() const noexcept ;
```

A *strided* layout has constant striding between multi-index coordinates. Let `A` be an `array_ref` and `indices_V...` and `indices_U...` be multi-indices in the domain space such that all coordinates are equal except for the *ith* coordinate where `indices_V[ith] = indices_U[ith] + 1`. Then `stride(ith) = distance(& A(indices_V...) - & A( indices_U... ))` is constant for all coordinates.

```
template< typename IntegralType >
constexpr size_type stride( IntegralType index ) const noexcept
```

Requires: `is_strided()`.

Returns: When `r < rank()` the distance between members when the index of coordinate `r` is incremented by one, otherwise 0.

2.2.7 Constructors, assignments, destructor

```
constexpr array_ref() noexcept
```

Effect: Construct a *null* `array_ref` with `data() == nullptr` and `extent(i) == 0` for all dynamic dimensions.

```
constexpr array_ref( array_ref const & rhs ) noexcept
```

Effect: Construct an `array_ref` of the same span of objects referenced by `rhs`.

Remark: There may be other `Properties...` dependent effects.

```
constexpr array_ref( array_ref && rhs ) noexcept
```

Effect: Construct an `array_ref` the span of objects referenced by `rhs` and then `rhs` is a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

```
array_ref & operator = ( array_ref const & ) noexcept
array_ref & operator = ( array_ref && rhs ) noexcept = default
```

Effect: `*this` has equal domain, equal codomain, and equivalent mapping.

Remark: There may be other `Properties...` dependent effects.

```
template< typename UType , typename ... UProperties >
constexpr array_ref( array_ref< UType , UProperties ... > const & ) noexcept
template< typename UType , typename ... UProperties >
array_ref & operator = ( array_ref< UType , UProperties ... > const & ) noexcept
```

Requires: Given using `V = array_ref<DataType,Properties...>` and using `U = array_ref<UType,UProperties...>` then

`is_assignable<V::value_type,U::value_type>` ,
`V::rank() == U::rank()` ,
`V::static_extent(r) == V::static_extent(r) OR V::static_extent(r) == 0` for `0 <= r < V::rank()` ,
compatibility of layout mapping, and
potentially other property compatibility conditions.

Effect: `* this` has equal domain, equal codomain, and equivalent mapping.

Remark: There may be other `Properties...` dependent effects.

```
template< typename ... IntegralExtent >
```

```
constexpr array_ref( pointer ptr , IntegralExtent ... dynamic_extent) noexcept
```

Requires: `conjunction<is_integral<IntegralExtent>::value...>::value.sizeof...(IntegralExtent) == rank()`. Each `dynamic_extent` is non-negative. The span of objects denoted by `[ ptr , ptr + required_span(dynamic_extent...) )`, shall be a valid contiguous span of objects.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents equal to the input `dynamic_extent...` and codomain equal to `[ ptr .. ptr + required_span(dynamic_extent...) )`

```
constexpr array_ref( pointer ptr , layout const& lay ) noexcept
```

Requires: The span of objects denoted by `[ ptr , ptr + required_span(lay) )`, shall be a valid contiguous span of objects.

Effects: This *wrapping constructor* constructs `* this` with domain's dynamic extents extracted from `lay` and codomain equal to `[ ptr .. ptr + required_span(dynamic_extent...) )`

```
~array_ref()
```

Effect: Assigns this to be a *null* `array_ref`.

Remark: There may be other `Properties...` dependent effects.

2.3 subarray

```
template< typename DataType , typename ... Properties , typename ... SliceSpecifiers >
```

```
detail::subarray_deduction_t<array_ref<DataType,Properties...>,SliceSpecifiers...>
```

```
subarray( array_ref< DataType, Properties ... > const & U , SliceSpecifiers ... S ) noexcept;
```

The `detail::subarray_deduction_t` is for exposition only to indicate that the implementation will require a metafunction to deduce the resulting `array_ref` type from the input `array_ref` and slice specifiers.

Let an *integral range* be denoted by any of the following.

- an `initializer_list<T>` of integral type τ and size 2
- a `pair<T,T>` of integral type τ
- a `tuple<T,T>` of integral type τ
- an `array<T,2>` of integral type τ
- `all` to denote `[0..extent(ith))`

Let the *ith* member of `s` be denoted by `S[ith]`.

Requires: `U.rank() == sizeof...(SliceSpecifiers)`. `S[ith]` is an *integral value* or an *integral range*. If `S[ith]` is an integral range then let `begin(S[ith])` be the beginning of the integral range `end(S[ith])` be the end of the integral range. If `S[ith]` is an integral value then let `begin(S[ith]) == S[ith]` and `end(S[ith]) == S[ith]+1`. `0 <= begin(S[ith]) <= end(S[ith]) <= A.extent(ith)`.

Returns: An `array_ref` `v` with a domain contained within the domain of `U`, codomain contained within the codomain of `U`, `v.rank()` is the number of integral ranges in `SlicedSpecifiers`, `U( begin(S)... )` refers to the same codomain member referred to by the mapping the zero-index of `v`, each integral value in `s...` contracts the corresponding extent of `U`.

Example:

```
// A.rank() == 4 and reference is lvalue reference
void foo( array_ref< DataType , Properties ... > const & A )
{
    auto B = subarray( A , make_pair(1,A.extent(0)-1) , 1 , make_pair(2,A.extent(2) , 2 );
    assert( & B(0,0) == A(1,1,2,2) );
    assert( & B(1,0) == A(2,1,2,2) );
    assert( & B(0,1) == A(1,1,3,2) );
}
```

3 Multidimensional Array Mapping and Member Access Properties

3.1 Header <array_ref> synopsis for properties

```

namespace std {
namespace experimental {

    // predefined layout mapping properties
    struct layout_right ;
    struct layout_left ;
    struct layout_stride ;

    template <typename T> struct is_layout ;
    template <typename T> constexpr bool is_layout_v = is_layout<T>::value;

    // extent size_type property
    template< typename T > struct extent_size_type ;

    // bounds checking property
    template< bool Enable >
    struct bounds_check_if ;

    using bounds_check = bounds_check_if< true > ;

    template< typename >
    struct is_array_property /* = std::integral_constant<bool,?> */ ;

    template< typename T >
    using is_array_property_v = is_array_property<T>::value ;

}}

```

3.2 Layout Mapping

An `array_ref` maps multi-indices from the domain to reference objects in the codomain by composing a *layout mapping* with a span of objects. The layout mapping is an extension point such that an `array_ref` may be instantiated with non-standard layout mappings.

3.2.1 Standard Layouts

The `layout_right` property denotes the C/C++ standard multidimensional array index mapping where the right-most extent is stride one and strides increase right-to-left as the product of extents. The `layout_left` property denotes the FORTRAN standard multidimensional array index mapping where the left-most extent is stride one and strides increase left-to-right as the product of extents. The `layout_stride` property denotes a multidimensional array index mapping with arbitrary strides for each extent.

The three standard layouts have the following layout mapping traits.

`layout_right`; i.e., the C/C++ standard layout

```

is_always_unique == true
is_always_contiguous == true
is_always_strided == true
When 0 < rank() then stride(rank()-1) == 1.
When 1 < rank() then stride(r-1) = stride(r) * extent(r) for 0 < r < rank() ..

```

For rank-two arrays (a.k.a., matrices) this is also known as *row major* layout.

`layout_left`; i.e., the FORTRAN standard layout

```

is_always_unique == true
is_always_contiguous == true
is_always_strided == true
When 0 < rank() then stride(0) == 1.
When 1 < rank() then stride(r) = stride(r-1) * extent(r-1) for 0 < r < rank() ..

```

For rank-two arrays (a.k.a., matrices) this is also known as *column major* layout.

`layout_stride`; i.e., an arbitrary **strided** layout

```

is_always_unique == false
is_always_contiguous == false
is_always_strided == true

```

3.2.2 Concept for Extensible Layout Mapping

A *layout* class conforms to the following interface such that an `array_ref` can compose the layout mapping with its `array_ref`

codomain member reference generation.

```
class layout_concept /* exposition only */ {
public:

    template< typename ExtentType , ExtentType ... >
    class mapping {
    public:

        // domain types

        using size_type = ExtentType ;

        // constructors, copy, assignment, and destructor

        constexpr mapping() noexcept;
        constexpr mapping(mapping&&) noexcept ;
        constexpr mapping(mapping const&) noexcept ;
        mapping& operator=(mapping&&) noexcept ;
        mapping& operator=(mapping const&) noexcept ;

        template <typename... IntegralExtent>
        explicit constexpr mapping( IntegralExtent... ) noexcept;

        explicit constexpr mapping( layout_concept const&) noexcept;

        ~mapping() noexcept ;

        // observers of domain: [0..extent(0)) X [0..extent(1)) X ... X [0..extent(rank()-1))

        static constexpr int rank() noexcept;
        static constexpr int rank_dynamic() noexcept;

        constexpr size_type size() const noexcept;

        constexpr size_type extent(int) const noexcept;

        constexpr size_type static_extent(int) noexcept;

        // observers of the codomain: [0..span())

        constexpr size_type span() const noexcept;

        template <typename... IntegralExtent>
        static constexpr size_type required_span( IntegralExtent ... ) noexcept;

        static constexpr size_type required_span( layout_concept const & ) noexcept;

        // observers of the mapping from domain to codomain

        static constexpr bool is_always_unique      = /* deduced */ ;
        static constexpr bool is_always_contiguous = /* deduced */ ;
        static constexpr bool is_always_strided     = /* deduced */ ;

        constexpr bool is_unique() const noexcept;
        constexpr bool is_contiguous() const noexcept;
        constexpr bool is_strided() noexcept;

        constexpr size_type stride(int) const noexcept;

        // mapping domain index to access codomain element

        template <typename... IntegralIndex>
        constexpr size_type operator()(IntegralIndex...) const noexcept;
    };
};
```

```
template< typename ExtentType , ExtentType ... Extent > class mapping
```

Requires: `is_integral<ExtentType>` and `Extent` is non-negative.

Effects: Defines the domain index space where `rank() == sizeof...(Extent)` and each `Extent == 0` denotes a dynamic dimension.

```
constexpr mapping() noexcept;
constexpr mapping(mapping&&) noexcept ;
constexpr mapping(mapping const&) noexcept ;
mapping& operator=(mapping&&) noexcept ;
```

```
mapping& operator=(mapping const&) noexcept ;
```

Customary constructors and assignment operators.

```
template <typename... IntegralExtent>
explicit constexpr mapping( IntegralExtent... ) noexcept;
explicit constexpr mapping( layout_concept const&) noexcept;

~mapping() noexcept ;
```

Constructors, assignment operators, and destructor requires and effects correspond to the corresponding members of `array_ref`.

```
static constexpr int rank() noexcept;
static constexpr int rank_dynamic() noexcept;
constexpr size_type size() const noexcept;
constexpr size_type extent(int) const noexcept;
constexpr size_type static_extent(int) noexcept;
constexpr size_type span() const noexcept;
```

```
template <typename... IntegralExtent>
static constexpr size_type required_span( IntegralExtent ... ) noexcept;
static constexpr size_type required_span( layout_concept const & ) noexcept;
```

```
static constexpr bool is_always_unique      = /* deduced */ ;
static constexpr bool is_always_contiguous = /* deduced */ ;
static constexpr bool is_always_strided    = /* deduced */ ;
```

```
constexpr bool is_unique() const noexcept;
constexpr bool is_contiguous() const noexcept;
constexpr bool is_strided() noexcept;
```

```
constexpr size_type stride(int) const noexcept;
```

Domain, codomain, and mapping observers requires and effects correspond to the corresponding members of `array_ref`.

```
template <typename... IntegralIndex>
constexpr size_type operator()(IntegralIndex...index) const noexcept;
```

Requires: `rank() == sizeof...(IntegralIndex)` and `0 <= index[i] < extent(i)`.

Returns: Layout mapping of `index...` to codomain.

3.3 Property: `extent_size_type< integral >`

```
template< typename integral > struct extent_size_type ;
```

Requires: `is_integral< integral >`. Specify `array_ref::size_type` as *integral*. If unspecified then `array_ref::size_type` is `size_t` ;.

3.4 Property: `bounds_check_if< bool >`

```
template< bool Enable > struct bounds_check_if ;
using bounds_check = bounds_check<true> ;
```

When `array_ref` Properties... includes `bounds_check_if<true>` then the mapping operators `array_ref::operator()` and `array_ref::operator[]` verify that each index is valid, `0 <= index[i] < extent(i)`. Verification failure shall be reported.

4 Undesirable Extent Mechanism (B) Proposal

The extents struct and dyn value are only required for the undesirable extents specification mechanism B.

```
namespace std {
namespace experimental {

    template< size_t ... IntegralExtent >
    struct extents {

        static constexpr int rank() noexcept;
```

```

static constexpr int rank_dynamic() noexcept;

static constexpr size_t static_extent(int) noexcept ;
};

constexpr size_t dyn = 0 ; // or ~size_t(0)
}}
```

Effects: | `IntegralExtent == dyn` indicates a dynamic extent. | `rank() == sizeof...(IntegralExtent)` | `rank_dynamic()` is the number of dynamic extents. | `static_extent(r) == IntegralExtent[r]`

5 Future Directions and Extensions

6 Revision History

6.1 P0009r0

Original multidimensional array reference paper with motivation, specification, and examples.

6.2 P0009r1

Revised with renaming from `view` to `array_ref` and allow unbounded rank through variadic arguments.

6.3 P0009r2

Adding details for extensibility of layout mapping.

Move motivation, examples, and relaxed incomplete array type proposal to separate papers.

- P0331 : Motivation and Examples for Polymorphic Multidimensional Array
- P0332 : Relaxed Incomplete Multidimensional Array Type Declaration

6.4 P0009r3

Oulu-2016 LEWG strawpoll: Move iterator from this paper to a subsequent paper.

Oulu-2016 LEWG feedback: <http://wiki.edg.com/bin/view/Wg21oulu/P0009>

- Array extents specification mechanism options are either-or, not both.
- List potential names for LEWG and/or LWG todo bikeshedding.
- Clearly & concisely note difference between multidimensional array versus language's array-of-array-of-array...
- Actual specification of reference type (and others), not "typically is" vagueness.
- Future directions / extensibility section regarding Properties...

The domain space specification *preferred* and *undesirable* mechanisms changed from accepting both to accepting only one.

Tighten up domain, codomain, and domain -> codomain mapping specifications.

Consistently use *extent* and *extents* for the multidimensional index space.

6.5 Future reversion

Align with and extend layout properties of P0454, Wording for Minimal `mdspan`.

7 References

ISOCPP issue: https://issues.isocpp.org/show_bug.cgi?id=80

7.1 P0454 : Wording for a Minimal `mdspan`

The `mdspan` is a minimal version of the multidimensional array view capability.

7.2 P0122 : `span`: bounds-safe views for sequences of objects

The `array_ref` codomain concept of *span* is well-aligned with this paper.

7.3 P0367 : Accessors

The P0367 Accessors proposal includes polymorphic mechanisms for accessing the memory an object or span of objects. The `Properties...` extension point in this proposal is intended to include such memroy access properties.